

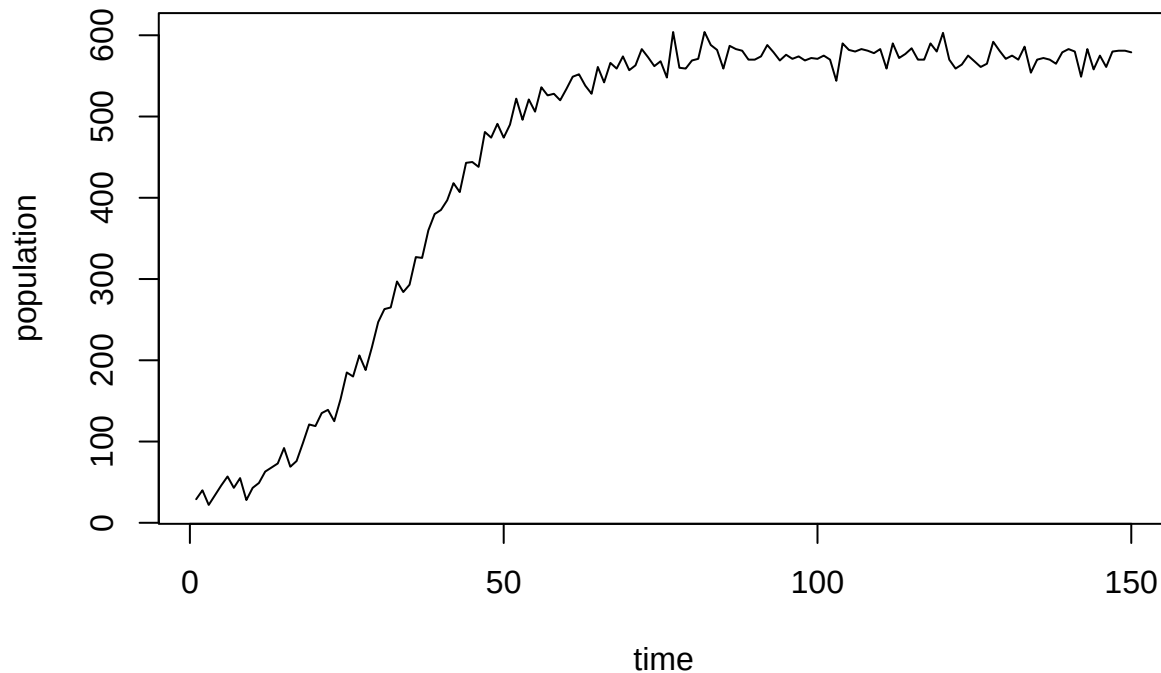
Introduction to Nonlinear Regression

Non-linear Regression

Example

Daphnia magna is a species of water flea native to North America, Asia, Europe, and Africa. Partly because it is easy and cheap to rear, it is a popular species for laboratory experiments in toxicology, ecology, and other fields. The plot below shows population numbers versus time measured for a population of daphnia raised under a certain laboratory protocol where the interest was in studying the population dynamics – that is, how the population numbers change over time in response to resource availability and other factors.

```
popdat<-read.table("daphinapop.txt")
names(popdat)<-c("time", "population")
attach(popdat)
plot(time,population,type="l")
```



We see that the population size initially expands rapidly, but then stabilizes at a fairly constant level. Our goal is to understand why this happens. We will use regression to better understand what is going on.

Regression of Daphnia Data

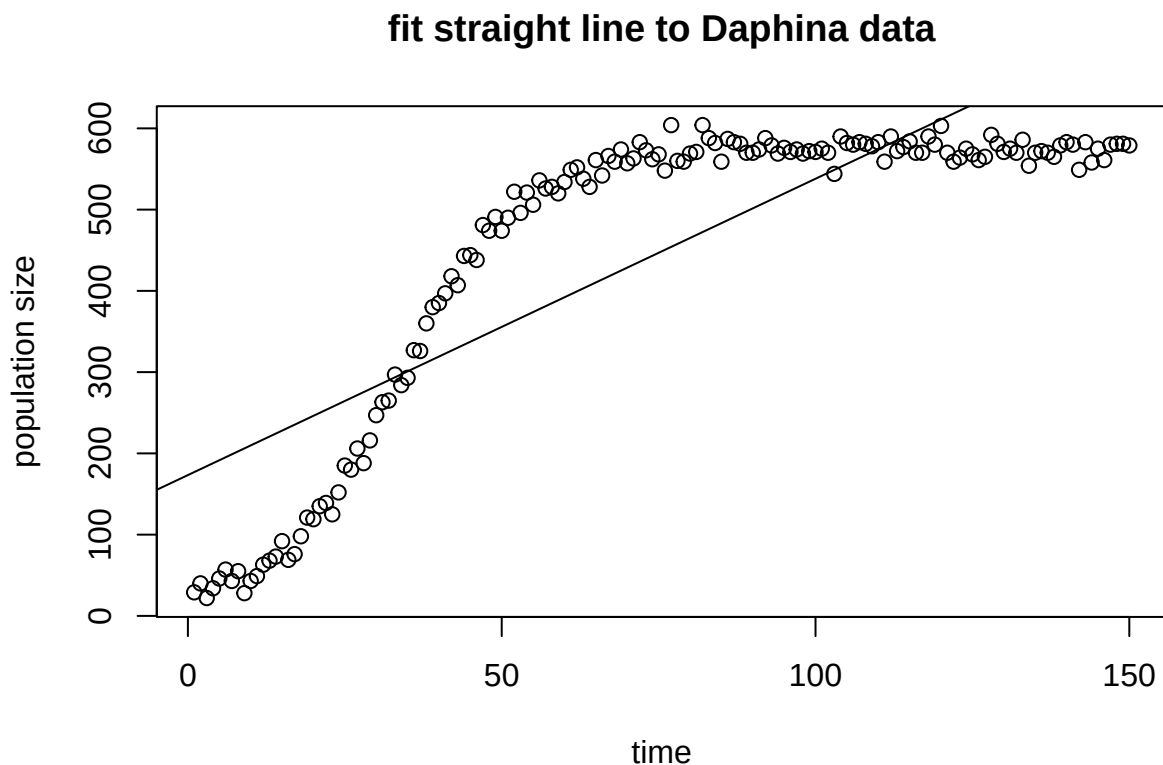
Let's start by fitting a straight line to the data:

```
poplm1<-lm(population~time,data=popdat)
summary(poplm1)
```

```
##
## Call:
## lm(formula = population ~ time, data = popdat)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -178.108 -101.129   5.278  101.705  159.071
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  173.2850    17.3788   9.971  <2e-16 ***
## time          3.6470     0.1997  18.265  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 105.9 on 148 degrees of freedom
## Multiple R-squared:  0.6927, Adjusted R-squared:  0.6906
## F-statistic: 333.6 on 1 and 148 DF,  p-value: < 2.2e-16
```

The regression is highly significant, but it does not do a good job of capturing the dynamics of the data:

```
plot(time,population, xlab="time",ylab="population size", main="fit straight line to Daphina data")
abline(coef(poplm1))
```



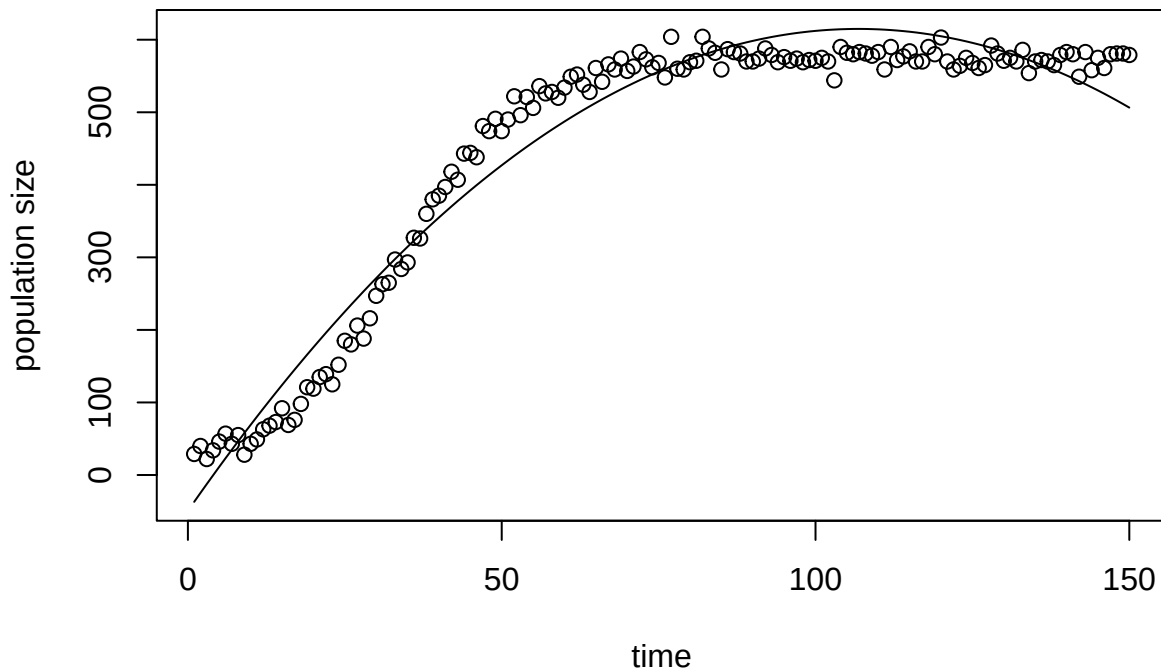
We can try fitting a polynomial:

```
poplm2<-lm(population~time+I(time^2),data=popdat)
summary(poplm2)
```

```
##
## Call:
## lm(formula = population ~ time + I(time^2), data = popdat)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -80.87 -32.40  -5.18  34.49  82.09
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) -49.357029   9.835662  -5.018 1.49e-06 ***
## time         12.435499   0.300731  41.351 < 2e-16 ***
## I(time^2)    -0.058202   0.001929 -30.170 < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 39.62 on 147 degrees of freedom
## Multiple R-squared:  0.9573, Adjusted R-squared:  0.9567
## F-statistic: 1647 on 2 and 147 DF,  p-value: < 2.2e-16
```

```
plot(time,fitted(poplm2),type="l", xlab="time",ylab="population size", main="fit 2nd degree polynomial to Daphnia data",
points(popdat[,2]))
```

fit 2nd degree polynomial to Daphnia data

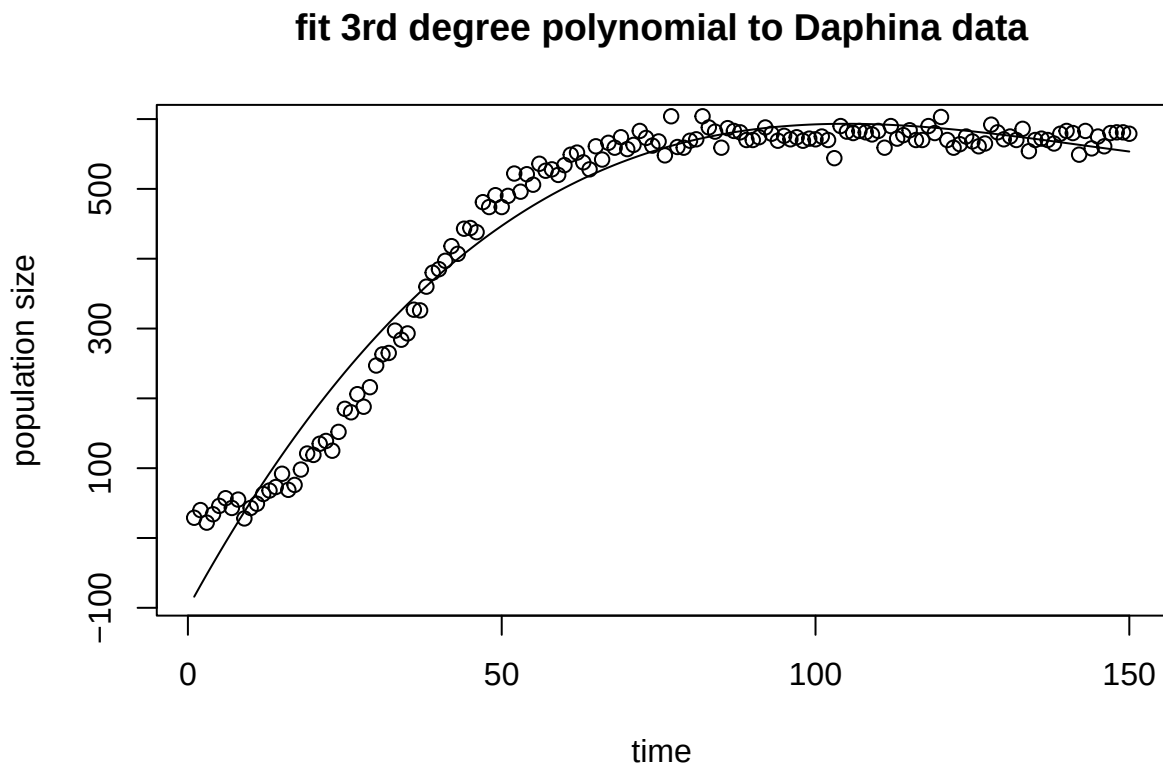


This is a little better, but is still clearly not capturing the true dynamics of the data. Let's try a third degree polynomial:

```
poplm3<-lm(population~time+I(time^2)+I(time^3),data=popdat)
summary(poplm3)
```

```
##
## Call:
## lm(formula = population ~ time + I(time^2) + I(time^3), data = popdat)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -90.305 -19.048  -1.618   23.156 113.081
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) -1.004e+02  1.174e+01  -8.553 1.49e-14 ***
## time         1.642e+01  6.709e-01  24.483 < 2e-16 ***
## I(time^2)    -1.240e-01  1.031e-02 -12.035 < 2e-16 ***
## I(time^3)     2.906e-04  4.487e-05   6.477 1.34e-09 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 35.04 on 146 degrees of freedom
## Multiple R-squared:  0.9668, Adjusted R-squared:  0.9661
## F-statistic: 1417 on 3 and 146 DF, p-value: < 2.2e-16
```

```
plot(time,fitted(poplm3),type="l",, xlab="time",ylab="population size", main="fit 3rd degree polynomial
points(popdat[,2])
```



This does somewhat better but is clearly not adequate. The use of polynomial requires that the curve turn

downwards or upwards at the end. However, this does not appear to be what is happening with the data. Thus, a polynomial model does not appear adequate.

Note: Non-linear regression refers to regression that is non-linear in the parameters. Thus, regression with a polynomial model

$$Y = \alpha + \beta_1 X + \beta_2 X^2 + \beta_3 X^3 + \dots + \epsilon$$

is linear regression because it is linear in the parameters even if not in the explanatory variables.

Exercise 1 Make a simulated data set from the model

$$f(x) = a + b * \sin(cx) + \epsilon$$

You can choose the parameter values. Make a plot similar to this:

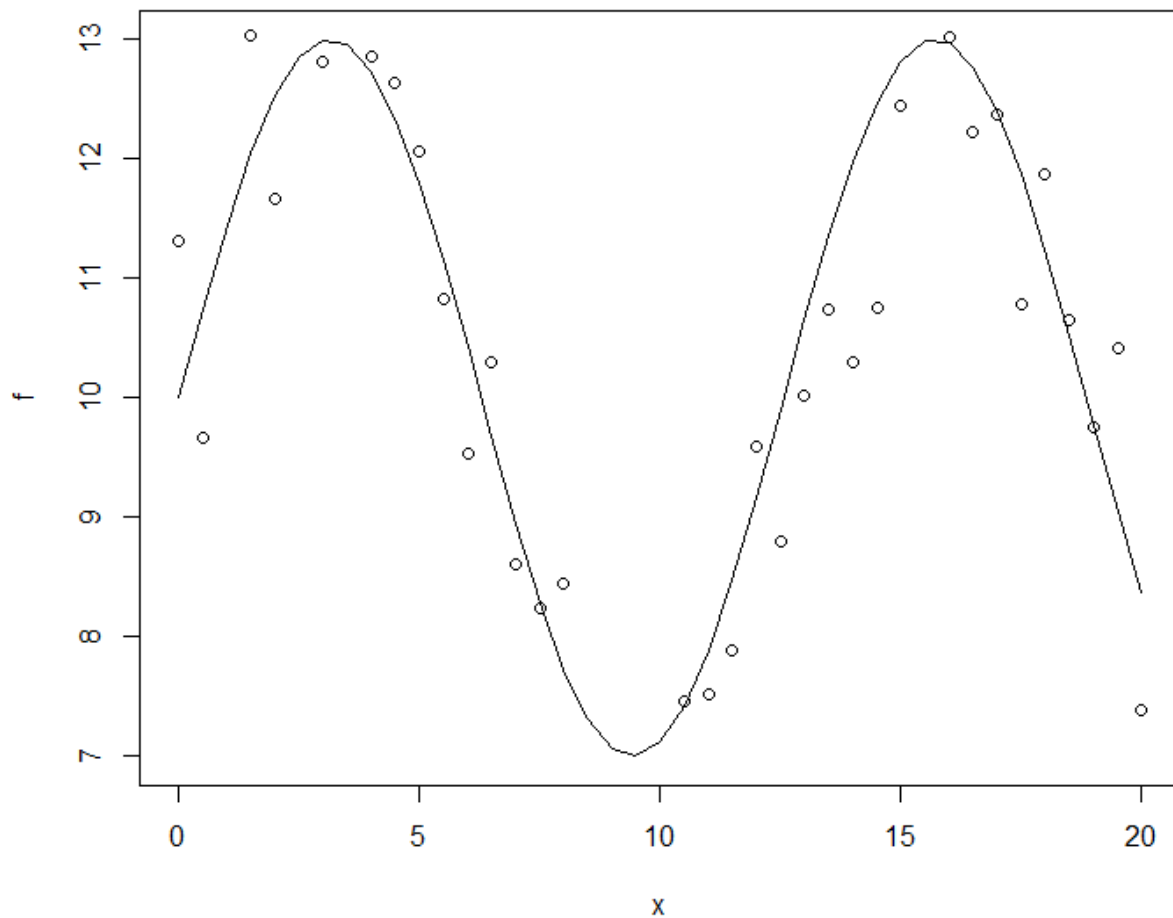


Figure 1: example plot for exercise

You will use this data later.

Some Ecological Theory

There is a very large and complex mathematical theory of population growth. However, this data looks like it will fit a very simple population growth model called the logistic growth model.

The logistic growth model is a differential equation:

$$\frac{dN}{dt} = rN \left(1 - \frac{N}{K} \right)$$

where N is population size, r is the maximum population growth rate, and K is the carrying capacity of the population. This is the maximum population that can be sustained given the available resources.

$\frac{dN}{dt}$ is the rate of change of the population with respect to time.

When the population size N is small compared to the carrying capacity K , then we have

$$\frac{dN}{dt} \approx rN$$

That is, the rate of growth of the population is equal to the population size multiplied by the population growth rate r . This equation can be easily solved:

$$N = N_0 e^{rt}$$

Where N_0 is the initial population size. That is, the population size increases exponentially.

However, as the population size increases, then N gets larger compared to K and the quantity $1 - \frac{N}{K}$ gets smaller. When $N=K$, then

$$1 - \frac{N}{K} = 1 - \frac{K}{K} = 1 - 1 = 0$$

Thus, we have

$$\frac{dN}{dt} = 0$$

That is, when the population is at its carrying capacity, then it stops growing and the population size is constant. This occurs because the population has reached the limit of the available resources and can no longer grow. Sometimes, the population will overshoot the resources (i.e. grow larger than the resources can support) and then the population size will crash. This can lead to boom-and-bust cycles in population size. The mathematical model above will not do this, but other population models will. Mathematical ecologists have spent a large amount of effort in figuring out the conditions when this occurs and relating the models to data.

The full differential equation has solution

$$N = \frac{K}{1 + \frac{K-N_0}{N_0} e^{-rt}}$$

You can prove this by plugging this into the differential equation above, but we won't do that here. Let's try fitting this to the *Daphnia* data.

Fitting non-linear regression models

The general form of a nonlinear regression model is

$$\vec{y} = f(\vec{x}, \vec{\beta}) + \epsilon$$

Where $\vec{x}$: an $n \times q$ dimensional data matrix (n is the sample size and q is the number of explanatory variables).
 β : a p -component vector of parameters

$$\epsilon \sim N(0, \sigma^2)$$

$f(\vec{X}, \vec{\beta})$: some non-linear function that gives the mean value of $\vec{y}$ as a function of the explanatory variables and the parameters.

Note: Unlike linear regression, it is not necessary that $p=q+1$.

Concepts of least squares estimation for linear models extend directly to non-linear regression. Take

$$Q = \sum_{i=1}^n \left[y_i - f(\vec{x}_i, \vec{\beta}) \right]^2$$

We minimize Q with respect to the parameter $\vec{\beta}$ in order to get parameter estimates. However, unlike with linear regression, there is no explicit solution to the maximization problem and so we must solve iteratively.

Most common computational methods use Taylor series:

$$f(\vec{x}_i, \vec{\beta}) \approx f(\vec{x}_i, \vec{\beta}^{(0)}) + \sum_{k=1}^p (\beta_k - \beta_k^{(0)}) \frac{\delta f}{\delta \beta_k} \Big|_{\beta_k = \beta_k^{(0)}}$$

That is, we approximate the nonlinear function $f(\vec{x}_i, \vec{\beta})$ with the linear function on the right-hand side.
 $\beta_j^{(0)}$ is an initial guess for the parameter vector and β_j is the true parameter vector.

We approximate the function $f(\vec{x}_i, \vec{\beta})$ in the neighborhood of $\beta_j^{(0)}$ using the usual Taylor series expansion.
The sum is over the different components of the parameter vector $\vec{\beta}$.

We can simplify the notation as follows:

$$\begin{aligned} f_i^{(0)} &= f(\vec{x}_i, \vec{\beta}) \\ \gamma_k^{(0)} &= \beta_k - \beta_k^{(0)} \\ D_{ik}^{(0)} &= \left[\frac{\delta f(\vec{x}_i, \vec{\beta})}{\delta \beta_k} \right]_{\vec{\beta} = \vec{\beta}^{(0)}} \end{aligned}$$

Then, we have

$$f(\vec{x}_i, \vec{\beta}) \approx f_i^{(0)} + \sum_{k=1}^p D_{ik}^{(0)} \gamma_k^{(0)}$$

Thus,

$$y_i = f(\vec{x}_i, \vec{\beta}) + \epsilon$$

becomes

$$y_i \approx f_i^{(0)} + \sum_{k=1}^p D_{ik}^{(0)} \gamma_k^{(0)} + \epsilon$$

We write this as

$$y_i - f_i^{(0)} = \sum_{k=1}^p D_{ik}^{(0)} \gamma_k^{(0)} + \epsilon$$

Or

$$y_i^{(0)} = \sum_{k=1}^p D_{ik}^{(0)} \gamma_k^{(0)} + \epsilon$$

Where

$$y_i^{(0)} = y_i - f_i^{(0)}$$

We then fit the $\gamma_k^{(0)}$ by linear regression. Where

$$\gamma_k^{(0)} = \beta_k - \beta_k^{(0)}$$

is the difference between the initial guess $\beta^{(0)}$ and the true parameter value β . Thus, we take a new value for β as

$$\beta_k^{(1)} = \beta_k^{(0)} + \gamma_k^{(0)}$$

That is, we use linear regression to estimate the difference between the true value and initial value. However, these new values are not least-squares estimates because the Taylor series approximation is not exact. However, if all works as it is supposed to, then $\beta_k^{(1)}$ should be closer to the true value β than $\beta_k^{(0)}$ was. We can then repeat the process and get closer still and continue iteratively until we converge to the final correct value.

This is known as the **Gauss-Newton method**. It is NOT guaranteed to converge (find a solution) and even if it does it is not guaranteed to find the correct solution, particularly if the initial values are chosen poorly. However, it will perform well in many situations.

Nonlinear Regression in R

The R function `nls` does nonlinear least squares.

```
?nls
```

```
## starting httpd help server ... done
```

Apply it to the daphnia data

```
N0<-popdat[1,2]
logrowth1<-nls(population ~ K/(1+(K-N0)/N0*exp(-r*time)),start=c(K=500,r=0.5),data=popdat)
summary(logrowth1)
```

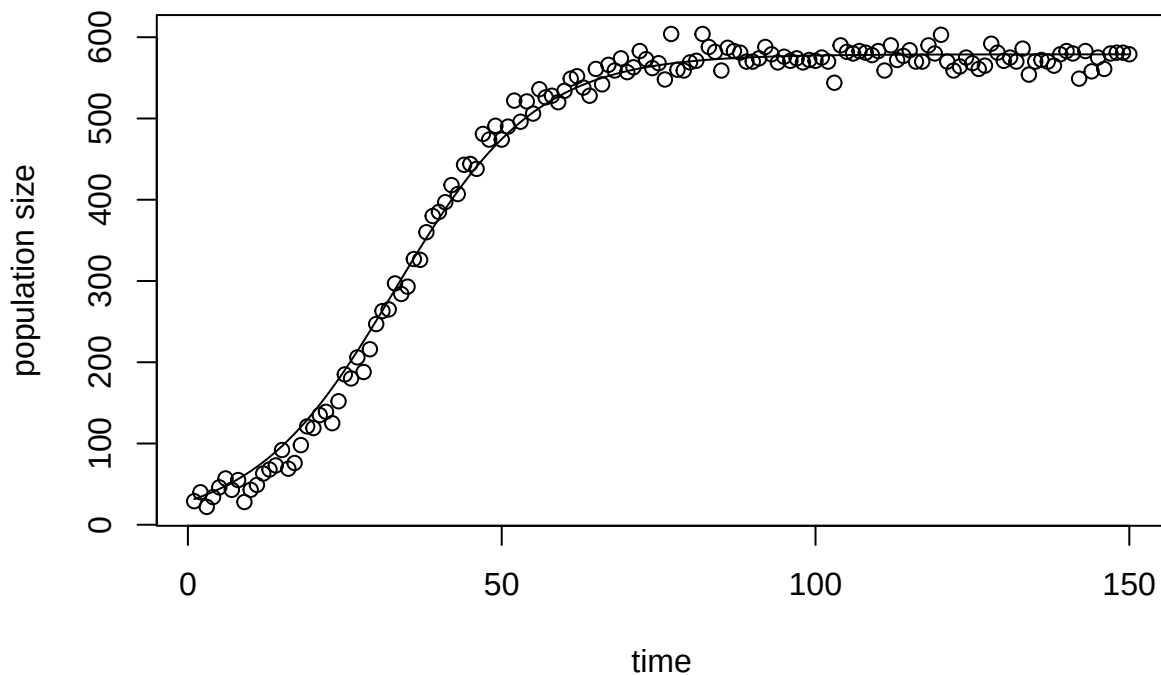


```
##
## Formula: population ~ K/(1 + (K - N0)/N0 * exp(-r * time))
##
## Parameters:
##      Estimate Std. Error t value Pr(>|t|)
## K 5.789e+02  1.637e+00   353.6  <2e-16 ***
## r 8.916e-02  5.747e-04   155.1  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 14.85 on 148 degrees of freedom
##
## Number of iterations to convergence: 8
## Achieved convergence tolerance: 4.432e-06
```

Thus, it estimates $K=573$ and $r=0.085$. The fitted curve is shown below:

```
plot(time,population, xlab="time",ylab="population size", main="fit logistic growth model to Daphina da
lines(fitted(loggrowth1))
```

fit logistic growth model to Daphina data



Note that fitting regression models to time series data is problematic because of autocorrelation (i.e. successive time points are not independent) and so the regression assumption of independence between data points is violated. We are ignoring this complication here (although we can in fact use `nls` for time series with some adjustments).

The convergence of `nls` can be improved by providing the derivatives of the regression function with respect to each of the parameters (otherwise, approximate derivatives are found via numerical methods). The derivatives are

$$df/dK = 1 / (1 + (K - N_0) / N_0 \exp(-rt)) - K \exp(-rt) / (1 + (K - N_0) / N_0 \exp(-rt))^2 / N_0$$

$$df/dr = K(K-N0)t \exp(-rt) / (1 + (K-N0)/N0 \exp(-rt))^2 / N0$$

We provide these as an attribute of the model

```
logmod<-function(K,r,N0,t)
{
  modfunc<-K/(1+(K-N0)/N0*exp(-r*t))
  derivK<-1/(1+(K-N0)/N0*exp(-r*t))-K*exp(-r*t)/(1+(K-N0)/N0*exp(-r*t))^2/N0
  derivr<-K*(K-N0)*t*exp(-r*t)/(1+(K-N0)/N0*exp(-r*t))^2/N0
  derivs<-cbind(derivK,derivr)
  #names(derivs)<-c("K", "r")
  attr(modfunc,"gradient")<-derivs
  return(modfunc)
}

logrowth2<-nls(population ~logmod(K,r,N0,time) ,start=c(K=500,r=0.5),data=popdat,trace=T)
```

```
##          4224250. (5.27e-01): par = (500 0.5)
## 2583923.   (1.03e+00): par = (491.2988 0.2387599)
## 747541.4   (4.23e+00): par = (506.6176 0.1250179)
## 426101.8   (3.76e+00): par = (569.0501 0.07157041)
## 42821.86   (5.60e-01): par = (570.438 0.08799137)
## 32634.08   (1.03e-02): par = (578.8479 0.08923013)
## 32630.85   (7.84e-04): par = (578.9527 0.08915845)
## 32630.84   (5.89e-05): par = (578.9453 0.0891639)
## 32630.84   (4.43e-06): par = (578.9459 0.08916349)
```

```
summary(logrowth2)
```

```
##
## Formula: population ~ logmod(K, r, N0, time)
##
## Parameters:
##      Estimate Std. Error t value Pr(>|t|)
## K 5.789e+02  1.637e+00   353.6   <2e-16 ***
## r 8.916e-02  5.747e-04   155.1   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 14.85 on 148 degrees of freedom
##
## Number of iterations to convergence: 8
## Achieved convergence tolerance: 4.429e-06
```

It only makes a small difference with this example, although sometimes it can make a larger one.

Exercise #2 Use the nls function to fit the model to your data from exercise #1.

Exercise #3 Make a plot that includes the true model, the data, and the fitted model from exercise #2.

Nonlinear Regression in Python

The `scipy.optimize.curve_fit` function is commonly used for nonlinear least squares in Python.

https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.curve_fit.html

I will run Python in R Markdown using the `reticulate` library

```
library(reticulate)
```

```
## Warning: package 'reticulate' was built under R version 4.4.2
```

```
#this tells it to use anaconda as the python environment:  
#use_condaenv("anaconda3") #laptop  
use_condaenv("base")      #office desktop  
  
#you can use conda_llist() to see what python environments you have available.
```

Load modules and read in the data:

```
import numpy as np  
import matplotlib.pyplot as plt  
from scipy.optimize import curve_fit  
import pandas as pd  
  
daphpop=pd.read_csv("daphinapop.txt",sep=None,names=["t","N"])
```

```
## <string>:3: ParserWarning: Falling back to the 'python' engine because the 'c' engine does not support
```

Similar to the case with R, we must define a python function that defines the mathematical function to be fit. The first argument must be the dependent variable and the remaining arguments must be the parameters to be fit.

```
NO=daphpop.iloc[0,1] #initial popvalue  
  
def log_growth(t,K,r):  
    return K/(1+(K-NO)/NO*np.exp(-r*t))
```

Now we use the `curve_fit` function to fit the parameters.

```
popt,pcov=curve_fit(log_growth,xdata=daphpop["t"],ydata=daphpop["N"],p0=[500,1])  
popt
```

```
## array([5.78945902e+02, 8.91634706e-02])
```

These are the same parameter estimates as with R.

Nonlinear regression when we don't know the correct model

We see in the above example that the logistic growth model fits the data well. However, while it may capture some of the dynamics of the population growth, it probably does not give the true relationship between time and population size. This is some complicated function that we will never know. This is a general problem with non-linear regression. We usually do not know what the true relationship is between variables. We might have some ideas, but it is unlikely that we know the exact relationship. We usually fall back on mathematically convenient models like the one above. Unfortunately, this can often lead us astray.

Another ecological example:

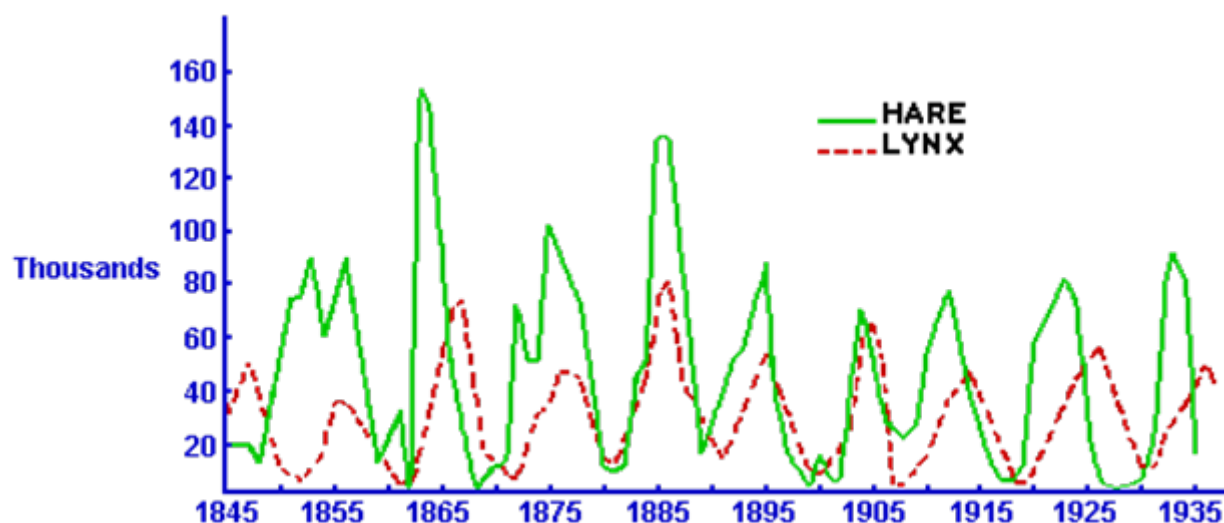


Figure 2: Lynx and Hare Data

The plot above shows a famous data set in population ecology. This shows the number of hare and lynx pelts traded by the Hudson Bay Company over a period of 100 years. We see a strong cyclic pattern in both populations, with a period of about 10 years. The lynx cycle lags behind the hare cycle by perhaps 1-2 years. The lynx is a major predator of the hare and the hare is a major food source for the lynx. The explanation for the observed pattern is that 1) in absence of many lynx, the hare population increases rapidly; 2) as the hare population increases, the lynx population also increases because there is plentiful food for the lynx; 3) As the lynx population begins to get large, they eat too many hare and the hare population collapses; 4) When the hare population collapses, the lynx population collapses soon after because there is not enough food; 5) Once the lynx population collapses, the hare population then starts to increase again. 6) Repeat.

Ecologists are interested in understanding what drives these dynamics. There has been a large amount of work in developing mathematical models for population dynamics and then fitting these models to data.

A standard predator-prey model has the form

$$\begin{aligned}\frac{dX}{dt} &= f_1(X) - f_2(X, Y) \\ \frac{dY}{dt} &= -g_1(Y) + g_2(X, Y)\end{aligned}$$

X is the prey population size

Y is the predator population size

$f_1(X)$ is the growth rate of the prey population in absence of the predator (e.g. the logistic growth function)

$f_2(X, Y)$ is the decrease in the prey population due to the predator

$-g_1(Y)$ is the (negative) growth rate of the predator population in absence of the prey (they die out with no

food)

$g_2(X, Y)$ is the increase in predators due to their predation on the prey

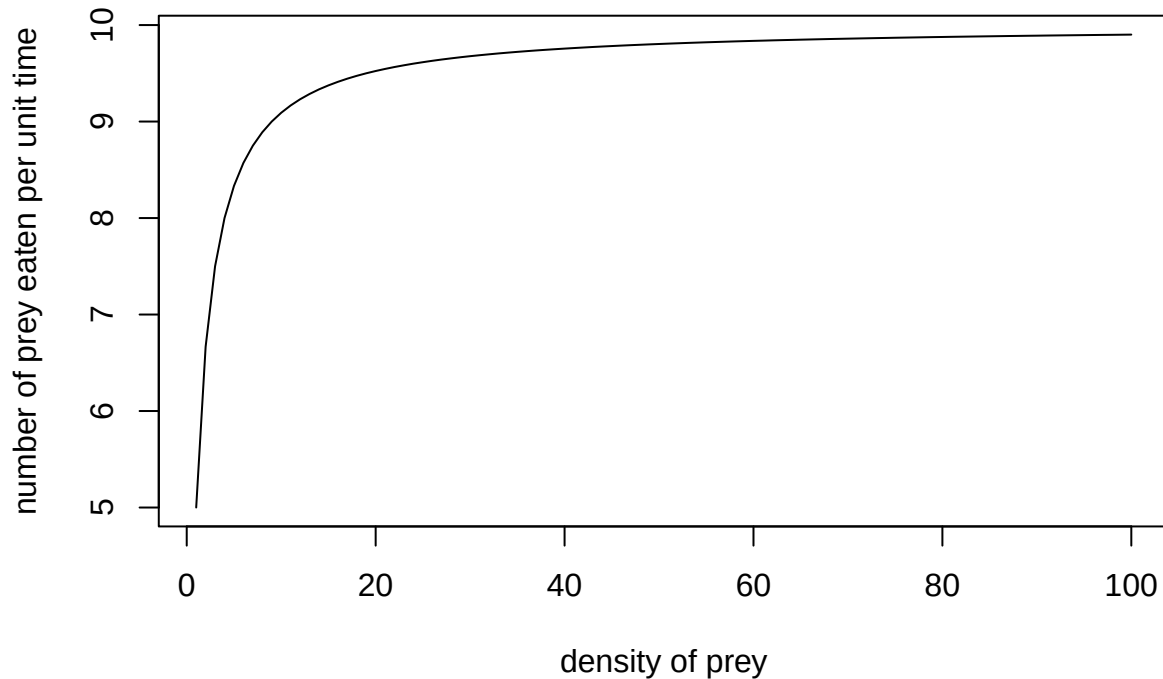
Typically, when people are interested in fitting one of these models to population data, they will use some standard functional form for the rate terms $f_1(X)$, $f_2(X, Y)$, etc.

For example, one popular choice for the predation term (known as the Holling type 2 functional response) is

$$f_2(X, Y) = \frac{\alpha_1 XY}{1 + \beta X}$$

Plot this

```
x<-1:100
y<-10
a<-1
B<-1
plot(x,a*x*y/(1+B*x), type="l",xlab="density of prey", ylab="number of prey eaten per unit time")
```



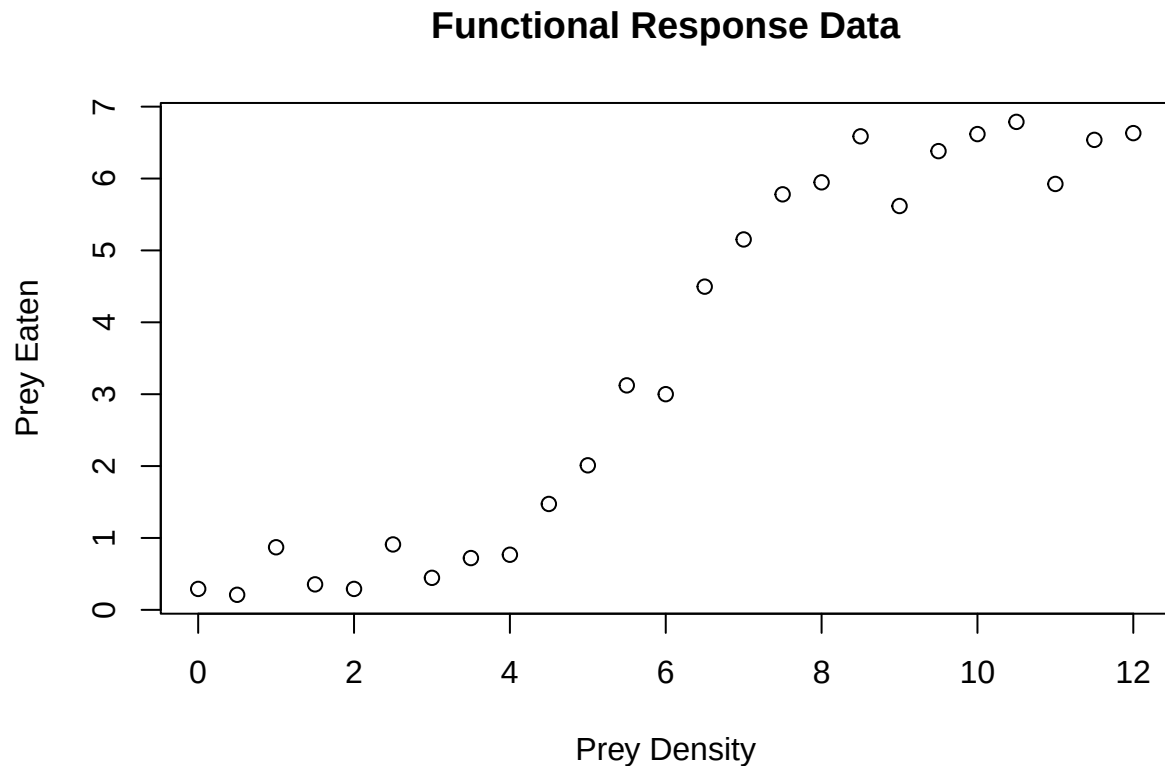
So, the amount eaten by each predator initially increases with the amount of prey X but flattens out as the prey density increases beyond the predator's ability to consume them.

The function $g_2(X, Y)$ has the same form but a different constant α_2 (with the ratio $\frac{\alpha_2}{\alpha_1}$ determining how efficiently prey biomass is converted into predator biomass).

$$g_2(X, Y) = \frac{\alpha_2 XY}{1 + \beta X}$$

Suppose that we are trying to fit a time series to a differential equation model like above. We have some external data available on the functional response. So, we want to fit the functional response using the external data and then use it in the differential equation model. The data in `FuncRest.txt` shows the results of an experiment looking at the predation functional response. It has two columns. The first is the prey density on some scale, which was set by the experimenters. The second column shows the average number of prey eaten by each predator per unit time.

```
dat1<-read.table("FuncResp.txt")
names(dat1)<-c("PreyDens","PreyEaten")
plot(dat1[,1],dat1[,2],xlab="Prey Density",ylab="Prey Eaten",main="Functional Response Data")
```



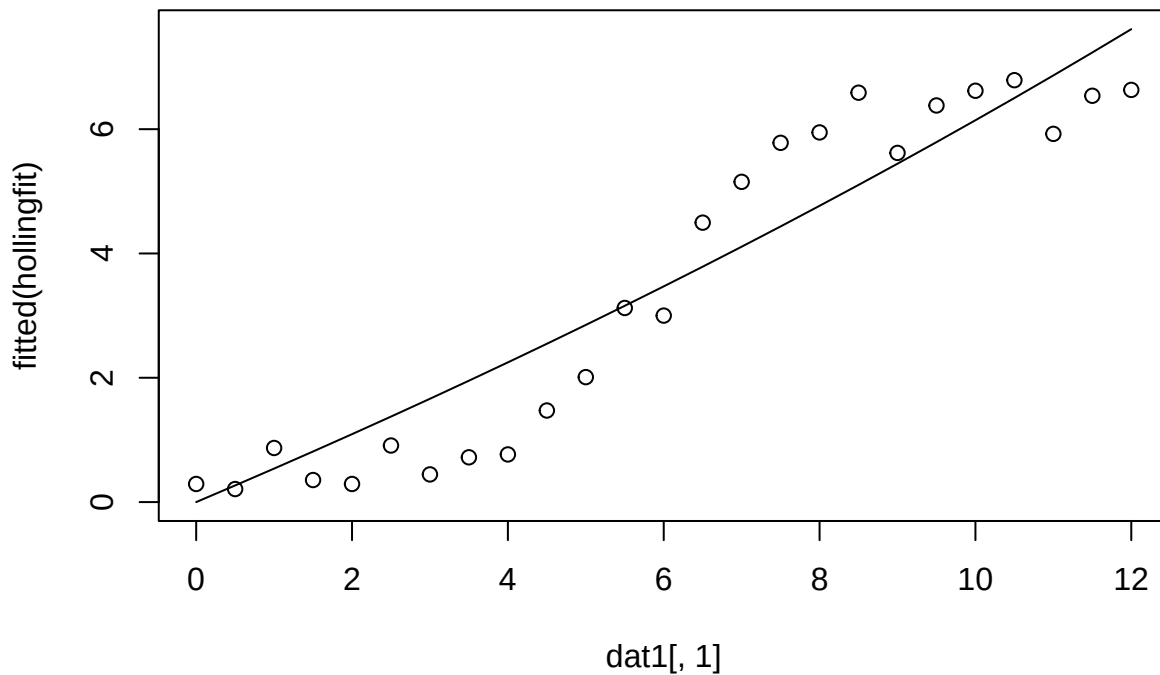
Let's try fitting this to the Holling functional response equation using the `nls` function:

We need initial values for the parameters. The curve should reach half maximum when $X = \frac{1}{\beta}$ and its maximum value should be about $\frac{\alpha_1}{\beta}$ (try it). The curve reaches half its maximum at about $X = 6$. Thus, we take $\beta = \frac{1}{6}$. Its maximum value is at about 7, so we take $\alpha_1 = \beta * 7 = 7/6$.

```
hollingfit<-nls(PreyEaten~a*PreyDens/(1+b*PreyDens),data=dat1,start=c(a=7/6,b=1/6))
summary(hollingfit)
```

```
##
## Formula: PreyEaten ~ a * PreyDens/(1 + b * PreyDens)
##
## Parameters:
##   Estimate Std. Error t value Pr(>|t|)
## a  0.53170    0.08601   6.182 2.63e-06 ***
## b -0.01345    0.01446  -0.930  0.362
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.898 on 23 degrees of freedom
##
## Number of iterations to convergence: 11
## Achieved convergence tolerance: 5.414e-06
```

```
plot(dat1[,1],fitted(hollingfit),type="l")
points(dat1)
```

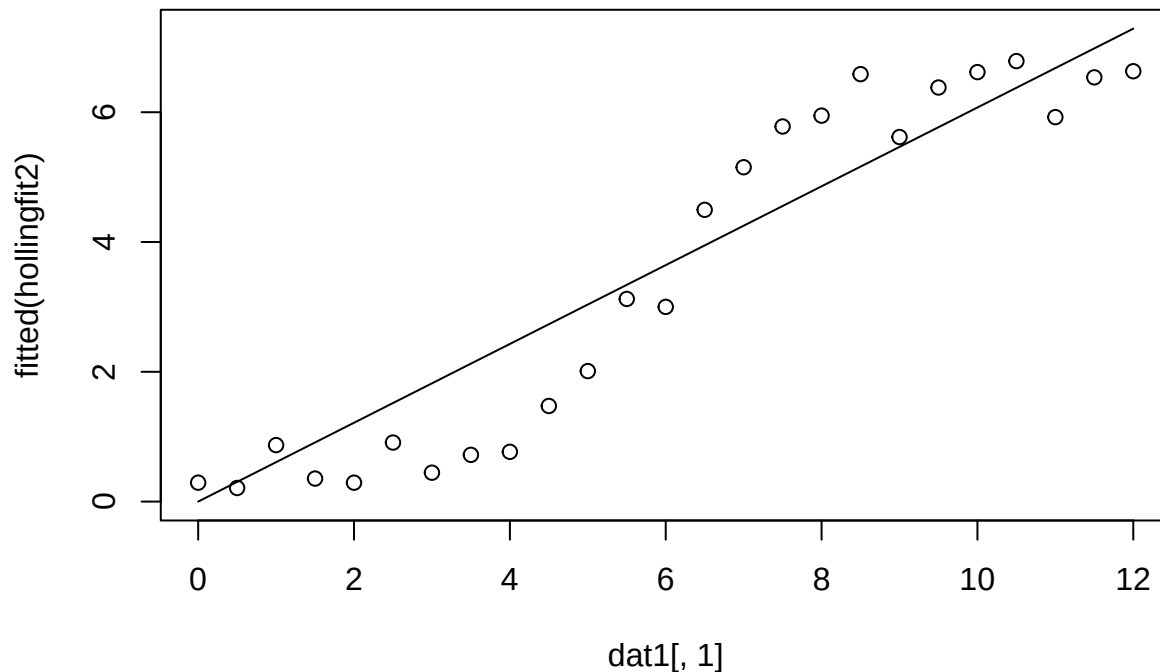


We see that the best fit for b is a negative number. However, both a and b must be positive. We can add a constraint using the lower and upper arguments with `nls`, but must also use the “port” algorithm (see `?nls`).

```
hollingfit2<-nls(PreyEaten~a*PreyDens/(1+b*PreyDens),data=dat1,start=c(a=7/6,b=0.167),lower=c(0,0),algo:
summary(hollingfit2)
```

```
##
## Formula: PreyEaten ~ a * PreyDens/(1 + b * PreyDens)
##
## Parameters:
##   Estimate Std. Error t value Pr(>|t|)
## a  0.60728    0.10487   5.791 6.71e-06 ***
## b  0.00000    0.01821   0.000      1
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.9167 on 23 degrees of freedom
##
## Algorithm "port", convergence message: both X-convergence and relative convergence (5)
```

```
plot(dat1[,1],fitted(hollingfit2),type="l")
points(dat1)
```

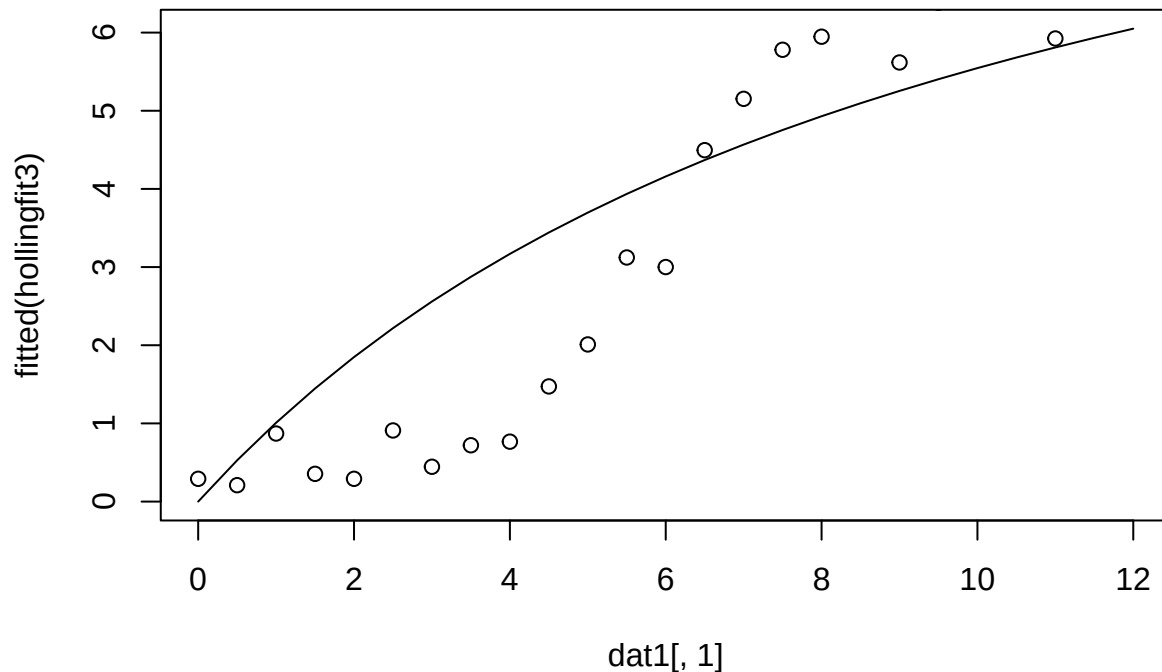


The best fit ends up with $b=0$, that is simply a straight line. We can try one more thing and constrain b in what appears to be the correct region:

```
hollingfit3<-nls(PreyEaten~a*PreyDens/(1+b*PreyDens),data=dat1,start=c(a=7/6,b=0.167),lower=c(0,0.1),upper=c(1,0.5))
summary(hollingfit3)
```

```
##
## Formula: PreyEaten ~ a * PreyDens/(1 + b * PreyDens)
##
## Parameters:
##   Estimate Std. Error t value Pr(>|t|)
## a  1.10915    0.33993   3.263  0.00342 **
## b  0.10000    0.06741   1.484  0.15150
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.285 on 23 degrees of freedom
##
## Algorithm "port", convergence message: both X-convergence and relative convergence (5)
```

```
plot(dat1[,1],fitted(hollingfit3),type="l")
points(dat1)
```

Again, it is trying to fit as close to a straight line as the constraint allows. The problem is that the Holling functional response is just not adequate. The Holling function increases linearly from 0, but the data appears to increase more slowly than linearly initially. This is presumably because when prey density is low the predators have a hard time finding any prey at all and the increasing density doesn't have much effect.

Now, we could try to think of another functional form that would capture this. However, if the goal is fitting the differential equation model above to the population dynamics, then we don't care about the functional response itself. All we care is that we fit a function that accurately reflects the true dynamics. Theory shows that the low density behavior of the functional response can be very important to the population dynamics. Thus if we get it wrong by trying to impose some functional form on the data it may make it impossible to get the population dynamics. This leads us to regression methods that don't depend on any model assumptions.